8 Configuring endpoint-level authorization: Applying restrictions

This chapter covers

- Selecting requests to apply restrictions using matcher methods
- Learning best-case scenarios for each matcher method

In chapter 7, you learned how to configure access based on authorities and roles. But we only applied the configurations for all endpoints. In this chapter, you'll learn how to apply authorization constraints to a specific group of requests. In production applications, it's less probable that you'll apply the same rules for all requests. You have endpoints that can be called only by specific users, while other endpoints might be accessible to everyone. Depending on the business requirements, each application has its own custom authorization configuration. Let's discuss the options available to refer to different requests when we write access configurations.

Even though we didn't pay attention to it, the first matcher method you used was the `anyRequest()` method. And because it was used in previous chapters, you know now that it refers to all requests, regardless of the path or HTTP method. It is the way to say "any request" or, sometimes, "any other request."

First, let's talk about selecting requests by path; then we can also add the HTTP method to the scenario. To choose the requests to which we apply authorization configuration, we use the `requestMatchers()` method.

8.1 Using the requestMatchers() method to select endpoints

In this section, you will learn how to use the `requestMatchers()` method in general so that in sections 8.2 through 8.4, we can continue describing various approaches to selecting HTTP requests for which you need to apply authorization restrictions. By the end of this chapter, you'll be able to apply the `requestMatchers()` method in any authorization configurations you might need to write for your application's requirements. Let's start with a straightforward example.

We create an application that exposes two endpoints: /hello and /ciao. We want to make sure that only users having the ADMIN role can call the /hello endpoint. Similarly, we want to make sure that only users having the MANAGER role can call the /ciao endpoint. You can find this example in the project ssia-ch8-ex1. The following listing defines the controller class.

Listing 8.1 The definition of the controller class

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public String hello() {
    return "Hello!";
  }

  @GetMapping("/ciao")
  public String ciao() {
    return "Ciao!";
  }
}
```

In the configuration class, we declare an `InMemoryUserDetailsManager` as our `UserDetailsService` instance and add two users with different roles. The user John has the ADMIN role, while Jane has the MANAGER role. To specify that only users having the ADMIN role can call the endpoint /hello when authorizing requests, we use the `requestMatchers()` method. The next listing presents the definition of the configuration class.

Listing 8.2 The definition of the configuration class

```
@Configuration
public class ProjectConfig {

  @Bean
  public UserDetailsService userDetailsService() {
    var manager = new InMemoryUserDetailsManager();

    var user1 = User.withUsername("john")
          .password("12345")
          .roles("ADMIN")
          .build();
```

```
    var user2 = User.withUsername("jane")
          .password("12345")
          .roles("MANAGER")
          .build();

    manager.createUser(user1);
    manager.createUser(user2);

    return manager;
  }

  @Bean
  public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.httpBasic(Customizer.withDefaults());

    http.authorizeHttpRequests(
      c -> c.requestMatchers("/hello").hasRole("ADMIN")       ①
          .requestMatchers("/ciao").hasRole("MANAGER")       ②
    );

    return http.build();
  }
}
```

① Only calls the path /hello if the user has the ADMIN role

② Only calls the path /ciao if the user has the MANAGER role

You can run and test this application. When you call the endpoint /hello with user John, you get a successful response. But if you call the same endpoint with user Jane, the response status returns an HTTP 403 Forbidden. Similarly, for the endpoint /ciao, you can only use Jane to get a successful result. For user John, the response status returns an HTTP 403 Forbidden. You can see the example calls using cURL in the code snippets that follow. To call the endpoint /hello for user John, use

```
curl -u john:12345 http://localhost:8080/hello
```

The response body is

```
Hello!
```

To call the endpoint /hello for user Jane, use

```
curl -u jane:12345 http://localhost:8080/hello
```

The response body is

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/hello"
}
```

To call the endpoint /ciao for user Jane, use

```
curl -u jane:12345 http://localhost:8080/ciao
```

The response body is

```
Ciao!
```

To call the endpoint /ciao for user John, use

```
curl -u john:12345 http://localhost:8080/ciao
```

The response body is

```
{
   "status":403,
   "error":"Forbidden",
   "message":"Forbidden",
```

```
    "path":"/ciao"
}
```

If you now add any other endpoint to your application, it is accessible by default to anyone, even unauthenticated users. Let's assume you add a new endpoint /hola as presented in the next listing.

Listing 8.3 Adding a new endpoint for path /hola to the application

```
@RestController
public class HelloController {

  // Omitted code

  @GetMapping("/hola")
  public String hola() {
    return "Hola!";
  }
}
```

When you access this new endpoint, you see that it is accessible with or without having a valid user. The code snippets that follow display this behavior. To call the endpoint /hola without authenticating, use

```
curl http://localhost:8080/hola
```

The response body is

```
Hola!
```

To call the endpoint /hola for user John, use

```
curl -u john:12345 http://localhost:8080/hola
```

The response body is

```
Hola!
```

You can make this behavior more visible if you like by using the `permitAll()` method. You do this by using the `anyRequest()` matcher method at the end of the configuration chain for the request authorization, as presented in listing 8.4.

NOTE It is good practice to make all your rules explicit. Listing 8.4 clearly and unambiguously indicates the intention to permit requests to endpoints for everyone, except for the endpoints /hello and /ciao.

Listing 8.4 Marking additional requests explicitly as accessible without authentication

```
@Configuration
public class ProjectConfig {

 // Omitted code

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http)
  throws Exception {

  http.httpBasic(Customizer.withDefaults());

  http.authorizeHttpRequests(
    c -> c.requestMatchers("/hello").hasRole("ADMIN")
      .requestMatchers("/ciao").hasRole("MANAGER")
      .anyRequest().permitAll()                    ①
  );

  return http.build();
 }
}
```

① The permitAll() method states that all other requests are allowed without authentication.

NOTE When you use matchers to refer to requests, the order of the rules should be from particular to general. This is why the `anyRequest()` method cannot be called before a more specific `requestMatchers()` method.

Unauthenticated vs. failed authentication

If you have designed an endpoint to be accessible to anyone, you can call it without providing a username and a password for authentication. In this case, Spring Security

won't do the authentication. If you, however, provide a username and a password, Spring Security evaluates them in the authentication process. If they are wrong (not known by the system), authentication fails, and the response status will be 401 Unauthorized. To be more precise, if you call the /hola endpoint for the configuration presented in listing 8.4, the app returns the body `Hola!` as expected, and the response status is 200 OK. For example:

```
curl http://localhost:8080/hola
```

The response body is

```
Hola!
```

However, if you call the endpoint with invalid credentials, the status of the response is 401 Unauthorized. In the next call, I use an invalid password:

```
curl -u bill:abcde http://localhost:8080/hola
```

The response body is

```
{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
  "path":"/hola"
}
```

This behavior might look strange, but it makes sense, as the framework evaluates any username and password if you provide them in the request. As you learned in chapter 7, the application always does authentication before authorization, as this figure shows.

The authorization filter allows any request to the /hola path. However, because the application first executes the authentication logic, the request is never forwarded to the authorization filter. Instead, the authentication filter replies with an HTTP 401 Unauthorized.

In conclusion, any situation in which authentication fails will generate a response with the status 401 Unauthorized, and the application won't forward the call to the endpoint. The `permitAll()` method refers to authorization configuration only, and if authentication fails, the call will not be allowed further.

You could decide, of course, to make all the other endpoints accessible only for authenticated users. To do this, you would change the `permitAll()` method with `authenticated()` as presented in the following listing. Similarly, you could even deny all other requests by using the `denyAll()` method.

Listing 8.5 Making other requests accessible for all authenticated users

```
@Configuration
public class ProjectConfig {

 // Omitted code

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http)
  throws Exception {

  http.httpBasic(Customizer.withDefaults());

  http.authorizeHttpRequests(
    c -> c.requestMatchers("/hello").hasRole("ADMIN")
      .requestMatchers("/ciao").hasRole("MANAGER")
      .anyRequest().authenticated()              ①
  );

  return http.build();
 }
}
```

① All other requests are accessible only by authenticated users.

You've become familiar with using matcher methods to refer to requests for which you want to configure authorization restrictions. Now we must go into more depth with the syntaxes you can use.

In most practical scenarios, multiple endpoints can have the same authorization rules, so you don't have to set them up endpoint by endpoint. Furthermore, you sometimes need to specify the HTTP method, not just the path, like we've done until now.

Other times, you only need to configure rules for an endpoint when its path is called with HTTP GET. In this case, you'd need to define different rules for HTTP POST and HTTP DELETE. In the next section, we take each type of matcher method and discuss these aspects in detail.

8.2 Selecting requests to apply authorization restrictions

In this section, we deep dive into configuring request matchers. Using the `requestMatchers()` method is a common approach to refer to requests for applying authorization configuration. Thus, I expect you to have many opportunities to use this method to refer to requests in the applications you develop.

This matcher uses the standard ANT syntax (table 8.1) for referring to paths. The syntax is the same as the one you use when writing endpoint mappings with annotations such as `@RequestMapping`, `@GetMapping`, `@PostMapping`, and so forth. The two methods you can use to declare MVC matchers are

- `requestMatchers(HttpMethod method, String... patterns)` —Lets you specify both the HTTP method to which the restrictions apply and the paths. This method is useful if you want to apply different restrictions for different HTTP methods for the same path.
- `requestMatchers(String... patterns)` —Simpler and easier to use if you only need to apply authorization restrictions based on paths. The restrictions can automatically apply to any HTTP method used with the path.

In this section, we approach multiple ways of using `requestMatchers()` methods. To demonstrate this, we start by writing an application that exposes multiple endpoints.

For the first time, we write endpoints that can be called with other HTTP methods besides GET. You might have observed that until now, I've avoided using other HTTP methods. The reason for this is that Spring Security by default applies protection against cross-site request forgery (CSRF). In chapter 9, we'll discuss how Spring Security mitigates this vulnerability by using CSRF tokens. But to make things simpler for the current example and to be able to call all endpoints, including those exposed with POST, PUT, or DELETE, we need to disable CSRF protection in our `securityFilterChain()` method:

```
http.csrf(
  c -> c.disable()
);
```

NOTE We disable CSRF protection now only to allow you to focus for the moment on the discussed topic: matcher methods. But don't rush to consider this a good approach. In chapter 9, we'll talk in detail about the CSRF protection provided by Spring Security.

We start by defining four endpoints to use in our tests:

- /a using the HTTP method GET
- /a using the HTTP method POST
- /a/b using the HTTP method GET
- /a/b/c using the HTTP method GET

With these endpoints, we can consider different scenarios for authorization configuration. The next listing provides the definitions of these endpoints. You can find this example in the project ssia-ch8-ex2.

Listing 8.6 Definition of the four endpoints for which we configure authorization

```
@RestController
public class TestController {

  @PostMapping("/a")
  public String postEndpointA() {
    return "Works!";
  }

  @GetMapping("/a")
  public String getEndpointA() {
    return "Works!";
  }

  @GetMapping("/a/b")
  public String getEnpointB() {
    return "Works!";
  }

  @GetMapping("/a/b/c")
  public String getEnpointC() {
    return "Works!";
  }
}
```

We also need a couple of users with different roles. To keep things simple, we continue using an `InMemoryUserDetailsManager`. In the next listing, you can see the definition of

the `UserDetailsService` in the configuration class.

Listing 8.7 The definition of the `UserDetailsService`

```
@Configuration
public class ProjectConfig {

 @Bean
 public UserDetailsService userDetailsService() {
  var manager = new InMemoryUserDetailsManager();    ①

  var user1 = User.withUsername("john")
        .password("12345")
        .roles("ADMIN")                    ②
        .build();

  var user2 = User.withUsername("jane")
        .password("12345")
        .roles("MANAGER")                  ③
        .build();

  manager.createUser(user1);
  manager.createUser(user2);

  return manager;
 }

 @Bean
 public PasswordEncoder passwordEncoder() {
  return NoOpPasswordEncoder.getInstance();        ④
 }
}
```

① Defines an InMemoryUserDetailsManager to store users

② User John has the ADMIN role.

③ User Jane has the MANAGER role.

④ Don't forget you also need to add a PasswordEncoder.

Let's start with the first scenario. For requests done using an HTTP GET method for the /a path, the application needs to authenticate the user. For the same path, requests us-

ing an HTTP POST method don't require authentication. The application denies all other requests. The following listing shows the configurations that you need to write to achieve this setup.

Listing 8.8 Authorization configuration for the first scenario, /a

```
@Configuration
public class ProjectConfig {

 // Omitted code

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http)
  throws Exception {

  http.httpBasic(Customizer.withDefaults());

  http.authorizeHttpRequests(
   c -> c.requestMatchers(HttpMethod.GET, "/a")
         .authenticated()                    ①
        .requestMatchers(HttpMethod.POST, "/a")
         .permitAll()                        ②
        .anyRequest()
         .denyAll()                          ③
  );

  http.csrf(
   c -> c.disable()
  );                                         ④

  return http.build();
 }
}
```

① For path /a requests called with an HTTP GET method, the app needs to authenticate the user.

② Permits path /a requests to be called with an HTTP POST method for anyone

③ Denies any other request to any other path

④ Disables CSRF to enable a call to the /a path using the HTTP POST method

In the code snippets that follow, we analyze the results of the calls to the endpoints for the configuration presented in listing 8.8. For the call to path /a using the HTTP method POST without authenticating, use this cURL command:

```
curl -XPOST http://localhost:8080/a
```

The response body is

```
Works!
```

When calling path /a using HTTP GET without authenticating, use

```
curl -XGET http://localhost:8080/a
```

The response is

```
{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
  "path":"/a"
}
```

If you want to change the response to a successful one, you need to authenticate with a valid user. For the following call

```
curl -u john:12345 -XGET http://localhost:8080/a
```

the response body is

```
Works!
```

However, user John isn't allowed to call path /a/b, so authenticating with his credentials for this call generates a 403 Forbidden:

```
curl -u john:12345 -XGET http://localhost:8080/a/b
```

The response is

```
{
 "status":403,
 "error":"Forbidden",
 "message":"Forbidden",
 "path":"/a/b"
}
```

With this example, you now know how to differentiate requests based on the HTTP method. But what if multiple paths have the same authorization rules? Of course, we can enumerate all the paths for which we apply authorization rules; however, if we have too many paths, this makes reading code uncomfortable. In addition, we might know from the beginning that a group of paths with the same prefix always has the same authorization rules. We want to make sure that adding a new path to the same group doesn't also change the authorization configuration. To manage these cases, we use *path expressions*. Let's prove these in an example.

For the current project, we want to ensure that the same rules apply for all requests for paths starting with /a/b. These paths in our case are /a/b and /a/b/c. To achieve this, we use the `**` operator. You can find this example in the project ssia-ch8-ex3.

Listing 8.9 Changes in the configuration class for multiple paths

```
@Configuration
public class ProjectConfig {

 // Omitted code

 @Bean
 public void configure(HttpSecurity http)
  throws Exception {

  http.httpBasic(Customizer.withDefaults());

  http.authorizeHttpRequests(
   c -> c.requestMatchers( "/a/b/**").authenticated()    ①
        .anyRequest().permitAll();
  );
```

```
    http.csrf(
      c -> c.disable()
    );

    return http.build();
  }
}
```

① The /a/b/** expression refers to all paths prefixed with /a/b.

With the configuration given in listing 8.9, you can call path /a without being authenticated, but for all paths prefixed with /a/b, the application needs to authenticate the user. The following code snippets present the results of calling the /a, /a/b, and /a/b/c endpoints. First, to call the /a path without authenticating, use

```
curl http://localhost:8080/a
```

The response body is

```
Works!
```

To call the /a/b path without authenticating, use

```
curl http://localhost:8080/a/b
```

The response is

```
{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
  "path":"/a/b"
}
```

To call the /a/b/c path without authenticating, use

```
curl http://localhost:8080/a/b/c
```

The response is

```
{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
  "path":"/a/b/c"
}
```

As presented in the previous examples, the `**` operator refers to any number of pathnames. You can use it as we have done in the last example so that you can match requests with paths having a known prefix. You can also use it in the middle of a path to refer to any number of pathnames or to refer to paths ending in a specific pattern, like `/a/**/c`. Therefore, `/a/**/c` would not only match /a/b/c but also /a/b/d/c and a/b/c/d/e/c and so on. If you only want to match one pathname, then you can use a single `*`. For example, `a/*/c` would match a/b/c and a/d/c but not a/b/d/c.

Because you generally use path variables, these can be useful to apply authorization rules for such requests. You can even apply rules referring to the path variable value. Do you remember the discussion from section 8.1 about the `denyAll()` method and restricting all requests?

Let's turn now to a more suitable example of what you have learned in this section. We have an endpoint with a path variable, and we want to deny all requests that use a value for the path variable that has anything else other than digits. You can find this example in the project ssia-ch8-ex4. The following listing presents the controller.

Listing 8.10 The definition of an endpoint with a path variable in a controller class

```
@RestController
public class ProductController {

  @GetMapping("/product/{code}")
  public String productCode(@PathVariable String code) {
    return code;
  }
}
```

The next listing shows how to configure authorization so that only calls that have a value containing only digits are always permitted, while all other calls are denied.

Listing 8.11 Configuring the authorization to permit only specific digits

```
@Configuration
public class ProjectConfig {

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.httpBasic(Customizer.withDefaults());

    http.authorizeHttpRequests(
      c -> c.requestMatchers("/product/{code:^[0-9]*$}")      ①
            .permitAll()
        .anyRequest()
            .denyAll()
    );

    return http.build();
  }
}
```

① The regex refers to strings of any length, containing any digit.

NOTE When using parameter expressions with a regex, make sure to not have a space between the name of the parameter, the colon (:), and the regex, as displayed in the listing.

Running this example, you can see the result as presented in the following code snippets. The application only accepts the call when the path variable value has digits only. To call the endpoint using the value `1234a`, use

```
curl http://localhost:8080/product/1234a
```

The response is

```
{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
```

```
  "path":"/product/1234a"
}
```

To call the endpoint with the value `12345`, use

```
curl http://localhost:8080/product/12345
```

The response is

```
12345
```

We extensively discussed and included plenty of examples of how to refer to requests using the `requestMatchers()` method. Table 8.1 is a refresher for the path expressions used in this section. You can refer to it later when you want to recall any of them.

Table 8.1 Common expressions used for path matching with MVC matchers

Expression	Description
`/a`	Only path /a.
/a/*	The `*` operator replaces one pathname. In this case, it matches /a/b or /a/c, but not /a/b/c.
`/a/**`	The `**` operator replaces multiple pathnames. In this case, /a, /a/b, and /a/b/c are a match for this expression.
/a/{param}	This expression applies to the path /a with a given path parameter.
`/a/{param:regex}`	This expression applies to the path /a with a given path parameter only when the value of the parameter matches the given regular expression.

8.3 Using regular expressions with request matchers

This section discusses regular expression (regex). You should already know what regular expressions are, but you don't need to be an expert on the subject. Any of the books recommended at **https://www.regular-expressions.info/books.html** are excellent resources from which you can learn about the subject in more depth. For writing regex, I also often use online generators such as **https://regexr.com/** (figure 8.1).

Figure 8.1 Letting your cat play on the keyboard is not the best solution for generating regular expressions (regex). To learn how to generate regexes, you can use an online generator such as **https://regexr.com/**.

Sections 8.2 and 8.3 showed that in most cases, it is possible use path expression syntaxes to refer to requests to which you apply authorization configurations. In some cases, however, you might have more particular requirements, and you cannot solve those with path expressions. An example of such a requirement could be, "Deny all requests when paths contain specific symbols or characters." For these scenarios, you need to use a more powerful expression like a regex.

You can use regexes to represent any format of a string, so they offer limitless possibilities for this matter. However, they have the disadvantage of being difficult to read, even when applied to simple scenarios. For this reason, you might prefer to use path expressions and fall back to regexes only when you have no other options. To implement a regex request matcher, you can use the `requestMatchers()` method with a `RegexRequestMatcher` implementation as parameter.

To show how regex matchers work, let's put them into action by building an application that provides video content to its users. The application that presents the video gets its content by calling the endpoint /video/{country}/{language}. For the sake of the example, the application receives the country and language in two path variables from where the user makes the request. We consider that any authenticated user can see the video content if the request comes from the United States, Canada, or the United Kingdom, or if they use English.

You can find this example implemented in the project ssia-ch8-ex5. The endpoint we need to secure has two path variables, as shown in the following listing. This makes the requirement complicated to implement with request matchers.

Listing 8.12 The definition of the endpoint for the controller class

```
@RestController
public class VideoController {

  @GetMapping("/video/{country}/{language}")
  public String video(@PathVariable String country,
                      @PathVariable String language) {
    return "Video allowed for " + country + " " + language;
  }
}
```

For a condition on a single path variable, we can write a regex directly in the path expression. We referred to such an example in section 8.2, but I didn't go into detail at that time because we weren't discussing regexes.

Let's assume you have the endpoint /email/{email}. You want to apply a rule using a matcher only to the requests that send an address ending in .com as a value of the `email` parameter. In that case, you write a request matcher as presented by the next code snippet. You can find the complete example in the project ssia-ch8-ex6:

```
http.authorizeHttpRequests(
  c -> c.requestMatchers("/email/{email:.*(?:.+@.+\\.com)}" ).permitAll()
        .anyRequest().denyAll();
);
```

If you test such a restriction, you can see that the application only accepts emails ending in .com. For example, to call the endpoint to jane@example.com, you can use

```
curl http://localhost:8080/email/jane@example.com
```

The response body is

```
Allowed for email jane@example.com
```

And to call the endpoint to jane@example.net, you use

```
curl http://localhost:8080/email/jane@example.net
```

The response body is

```
{
  "status":401,
  "error":"Unauthorized",
  "message":"Unauthorized",
  "path":/email/jane@example.net
}
```

It is fairly easy and makes it even clearer why we encounter regex matchers less frequently. However, as I said earlier, requirements are sometimes complex. You'll find it handier to use regex matchers when you find something like the following:

- Specific configurations for all paths containing phone numbers or email addresses
- Specific configurations for all paths having a certain format, including what is sent through all the path variables

Back to our regex matchers example (ssia-ch8-ex6): when you need to write a more complex rule, eventually referring to more path patterns and multiple path variable values, it's easier to write a regex matcher. Listing 8.13 presents the definition for the configuration class that uses a regex matcher to solve the requirement given for the /video/{country}/{language} path. We also add two users with different authorities to test the implementation.

Listing 8.13 The configuration class using a regex matcher

```
@Configuration
public class ProjectConfig {

  @Bean
  public UserDetailsService userDetailsService() {
    var uds = new InMemoryUserDetailsManager();

    var u1 = User.withUsername("john")
              .password("12345")
              .authorities("read")
              .build();

    var u2 = User.withUsername("jane")
              .password("12345")
              .authorities("read", "premium")
              .build();
```

```
    uds.createUser(u1);
    uds.createUser(u2);

    return uds;
  }

  @Bean
  public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.httpBasic(Customizer.withDefaults());

    http.authorizeHttpRequests(

      c -> c.regexMatchers(".*/(us|uk|ca)+/(en|fr).*")    ①
            .authenticated()
          .anyRequest()
            .hasAuthority("premium");              ②

    );

  }
}
```

① We use a regex to match the paths for which the user only needs to be authenticated.

② Configures the other paths for which the user needs to have premium access

Running and testing the endpoints confirm that the application applied the authorization configurations correctly. The user John can call the endpoint with the country code `US` and language `en`, but he can't call the endpoint for the country code `FR` and language `fr` due to the restrictions we configured. Calling the /video endpoint and authenticating user John for the US region and the English language looks like this:

```
curl -u john:12345 http://localhost:8080/video/us/en
```

The response body is

```
Video allowed for us en
```

Calling the /video endpoint and authenticating user John for the FR region and the French language looks like this:

```
curl -u john:12345 http://localhost:8080/video/fr/fr
```

The response body is

```
{
  "status":403,
  "error":"Forbidden",
  "message":"Forbidden",
  "path":"/video/fr/fr"
}
```

Having premium authority, user Jane makes both calls with success. For the first call

```
curl -u jane:12345 http://localhost:8080/video/us/en
```

the response body is

```
Video allowed for us en
```

For the second call

```
curl -u jane:12345 http://localhost:8080/video/fr/fr
```

the response body is

```
Video allowed for fr fr
```

Regexes are powerful tools. You can use them to refer to paths for any given requirement. However, because regexes are hard to read and can become quite long, they

should remain your last choice. Use them only if path expressions don't offer you a solution to your problem.

In this section, I used the simplest example I could imagine so that the needed regex would be short. But in more complex scenarios, the regex can become much longer. Of course, you'll find experts who say any regex is easy to read. For example, a regex used to match an email address might look like the one in the next code snippet. Can you easily read and understand it?

```
(?:[a-z0-9!#$%&'*+/=?^_`{|}~-]+(?:\.[a-z0-9!#$%&'*+/=?^_`{|}~-
]+)*|"(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f]|\\[\x01-
\x09\x0b\x0c\x0e-\x7f])*")@(?:(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.)+[a-z0
-9](?:[a-z0-9-]*[a-z0-9])?|\[(?:(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0
-9]?)\.){3}(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?|[a-z0-9-]*[a-z0
-9]:(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f]|\\[\x01-
\x09\x0b\x0c\x0e-\x7f])+)\])
```

Summary

- In real-world scenarios, different authorization rules are applied for different requests.
- The requests for which authorization rules are configured are specified based on the path and HTTP method. To do this, you use the `requestMatchers()` method.
- When requirements are too complex to be solved with path expressions, you can implement them using more powerful regexes.

Previous chapter
7 Configuring endpoint-level authorization: Restricting access

Next chapter
9 Configuring CSRF protection